فحوصات جودة البيانات في FunnelStream: الـ 66 فحص بالتفصيل

الملف ده بيشرح كل فحص بيشتغل في الـ Gold build. التعريفات موجودة في `warehouse/checks.py`. النتايج المذكورة في الآخر جاية من الـ Gold build على الـ scratch catalog يوم 13 أكتوبر 2026.

الفكرة ببساطة: بعد ما نبني جداول Gold، بنشغّل 66 سؤال على البيانات. كل سؤال بيرجّع رقم. في أسئلة لو جوابها مش صفر الـ build بيقف، وفي أسئلة بترجّع رقم بس عشان إنسان يقراه.

---

1. إزاي الفحوصات بتشتغل

الفحوصات هي آخر خطوة في الـ `scripts/run_batch.py` (Gold build). بعد ما Databricks يبني جداول Gold من نسخة Silver، الـ build بيشغّل كل فحص كأنه SQL statement واحد للقراءة بس، وبيرجّع رقم واحد.

- فحص من نوع **fail** بيرجّع عدد المخالفات. أي قيمة غير الصفر بتفشّل الـ batch.
- فحص من نوع **report** بيرجّع رقم بيتطبع ومش بيفشّل الـ batch أبدًا. هو موجود عشان إنسان يقراه.
- الفحص اللي ميقدرش يشتغل (query مكسور، أو manifest مش مقروء) ده خطأ تقني وبيوقّف الـ build. أما الفحص اللي اشتغل ولقى مخالفة فمبيعملش error: القايمة كلها بتفضل ظاهرة في النتيجة.
- الـ SQL مكتوب في Python، وبيستخدم تلات placeholders بس (`{silver}` و `{gold}` وأزواج المدينة والمنطقة المسموحة)، وفيه statement واحد ومفيش أي كلمة كتابة. اختبار (`tests/test_checks.py`) بيفحص كل query على القواعد دي.

ليه 66؟ القايمة في `warehouse/checks.py` فيها 64 فحص. مقارنة الـ manifest بتضيف اتنين، فالتشغيل بـ `--manifest` بيبلّغ عن 66. التشغيل من غير الـ manifest بيبلّغ عن 65، لأن فحص الـ manifest بيبقى وقتها entry واحد "skipped".

المجموعة	عدد الفحوصات	النوع	السؤال اللي بتجاوب عليه
Keys	25	24 fail و 1 report	كل صف unique؟ وكل foreign key بيشاور على صف حقيقي؟
Reconciliation	7	fail	Gold و Silver متفقين؟ والمجاميع المالية بتطلع مظبوطة؟
Silver hygiene	7	fail	فيه حاجة عدّت من الـ Silver validator وهي مش المفروض؟
Hierarchy and history	7	fail	الـ districts جوه مدنها؟ والـ slowly changing dimensions متّسقة؟
Reported	18	report	إيه النسب وأعداد الصفوف في التشغيل ده؟

المجموعة	عدد الفحوصات	النوع	السؤال اللي بتجاوب عليه
Manifest	2	1 fail و 1 report	الـ pipeline مسك الـ noise اللي الـ simulator حقنه؟
الإجمالي	**66**	**46 fail و 20 report**	

يعني إيه ده عمليًا؟ 46 فحص يقدروا يوقّفوا الـ batch. و 20 قياسات لإنسان يقراها ومبيفشلوش أبدًا. لما نقول "66 فحص، صفر فشل"، ده معناه إن ولا واحد من الـ 46 فحص من نوع fail لقى مخالفة، والـ 20 التانيين طلّعوا أرقامهم.

الحدود (thresholds) اللي الفحوصات بتستخدمها

الإعداد	القيمة	بيستخدمه
Future tolerance	5 دقايق (فرق الساعة اللي الـ Silver validator بيسمح بيه)	فحص 34
Line total tolerance	EGP 0.01 (تقريب)	فحص 37
Open order age	14 يوم	فحص 51
Manifest tolerance	2% من عدد الـ quarantine المتوقع	فحص 65

2. المجموعة 1: Keys (الفحوصات من 1 لـ 25)

ببساطة: زي رقم البطاقة. لو نفس الشخص اتسجّل مرتين، كل عدّ بعد كده هيبقى غلط. فحص الـ key بيحمي كل رقم بعده: لو order ظهر مرتين، كل revenue وكل order count بيشمله هيطلع أعلى من الحقيقة.

2.1 Duplicate keys في نسخة Silver (فحوصات 1 لـ 7)

كل فحص بيجمّع الجدول حسب الـ key بتاعه وبيعدّ قيم الـ key اللي ظهرت في أكتر من صف. القيمة المتوقعة 0.

#	الفحص	الـ Key	ليه مهم
1	`dup_key.silver_events`	`event_id`	صفّين لـ event واحد هيعدّوا الـ sessions والـ views والـ carts مرتين. ده الفحص بتاع إلغاء تكرار الـ 1% events المكررة.
2	`dup_key.silver_orders`	`order_id`	الـ order الواحد لازم يبقى صف واحد.
3	`dup_key.silver_order_status`	`event_id`	صف واحد لكل event تغيير حالة.
4	`dup_key.silver_order_items`	`order_id` , `line_no`	صف واحد لكل سطر في الـ order. السطر المكرر بيضخّم الـ revenue.

#	الفحص	الـ Key	ليه مهم
5	`dup_key.silver_products`	`product_id` , `updated_at`	صف واحد لكل نسخة من الـ product.
6	`dup_key.silver_customers`	`customer_id` , `updated_at`	صف واحد لكل نسخة من الـ customer.
7	`dup_key.silver_campaign_snapshots`	`campaign_id` , `report_date` , `pulled_at`	صف واحد لكل campaign في اليوم في كل سحبة. سحب الـ ad API مرتين مينفعش يكرّر صفوف.

2.2 Duplicate keys في جداول Gold (فحوصات 8 لـ 19)

نفس الفحص على الـ 12 جدول Gold. القيمة المتوقعة 0.

#	الفحص	الـ Key	ليه مهم
8	`dup_key.dim_customer`	`customer_key`	الـ surrogate key المكرر بيخلّي كل صف fact يعمل join مرتين.
9	`dup_key.dim_product`	`product_key`	نفس الكلام.
10	`dup_key.dim_campaign`	`campaign_key`	نفس الكلام.
11	`dup_key.dim_date`	`date_key`	نفس الكلام.
12	`dup_key.fact_orders`	`order_id`	صف واحد لكل order في جدول الـ fact.
13	`dup_key.fact_order_lines`	`order_id` , `line_no`	صف واحد لكل سطر order.
14	`dup_key.fact_attribution`	`order_id` , `model`	صف واحد لكل order ولكل attribution model (last touch و first touch).
15	`dup_key.agg_orders_daily`	`date_key` , `channel` , `payment_method` , `platform`	كل daily bucket موجود مرة واحدة، فصفحة Executive مبتعدّش مرتين.
16	`dup_key.agg_funnel_daily`	`event_date` , `country` , `channel` , `platform`	كل funnel bucket موجود مرة واحدة.
17	`dup_key.agg_campaign_daily`	`campaign_id` , `report_date` , `attribution_model`	كل campaign في اليوم والـ model موجود مرة واحدة، فالـ spend مبيتعدّش مرتين.
18	`dup_key.agg_city_product_daily`	`city` , `product_id` , `date_key`	كل city و product في اليوم موجود مرة واحدة.
19	`dup_key.customer_360`	`customer_id`	صف واحد لكل customer.

2.3 Orphan keys (فحوصات 20 لـ 24)

الـ orphan هو صف fact بيشاور على صف dimension أو صف أب مش موجود. الـ `-1` key هو "العضو المجهول" المسموح: الصف اللي بيشاور على `-1` مش orphan. أما الـ key اللي قيمته NULL فهو مخالفة دايمًا، لأن الـ build المفروض كان يكتب `-1`.

ببساطة: زي فاتورة مكتوب عليها اسم عميل مش موجود في السجل.

#	الفحص	بيعدّ إيه	ليه مهم
20	`orphan.fact_orders.customer_key`	صفوف `fact_orders` اللي `customer_key` فيها NULL أو مش موجود في `dim_customer`	الـ orders لازم تتبع customer معروف أو العضو المجهول، وإلا تقارير الـ customers هتخسر revenue.
21	`orphan.fact_order_lines.customer_key`	صفوف `fact_order_lines` بنفس المشكلة	نفس الكلام لسطور الـ order.
22	`orphan.fact_order_lines.product_key`	صفوف `fact_order_lines` اللي `product_key` فيها NULL أو مش موجود في `dim_product`	الـ revenue لازم دايمًا يلاقي product.
23	`orphan.fact_order_lines.order_id`	سطور order الـ order بتاعها مش موجود في `fact_orders`	سطر من غير order هيضيف revenue من غير order.
24	`orphan.fact_attribution.order_id`	صفوف attribution الـ order بتاعها مش موجود في `fact_orders`	الـ attribution لازم يوصف orders حقيقية.

2.4 Report: نسبة الـ customer المجهول (فحص 25)

`unknown_customer_share` هي نسبة صفوف `fact_orders` اللي ليها customer key مجهول `-1`، من 0 لـ 1. ده فحص من نوع report. القيمة العالية معناها إن orders كتير مقدرناش نطابقها بنسخة customer كانت سارية وقت الـ order.

---

3. المجموعة 2: Reconciliation (فحوصات 26 لـ 32)

ببساطة: زي ما تقارن مجموع الفاتورة بمجموع سطورها، ودفتر المحل بدفتر المخزن. لازم يتطابقوا.

فحوصات الـ reconciliation بتقارن جدول بجدول تاني المفروض يتفقوا معاه. بتمسك الـ build اللي بيضيّع أو بيخترع صفوف أو فلوس من غير ما حد يلاحظ.

#	الفحص	بيختبر إيه	ليه مهم
26	`recon.order_total_sum`	1 لو مجموع `fact_orders.order_total` بيختلف عن مجموع `fact_order_lines.line_total`، وإلا 0	فحص الفلوس السريع على قاعدة البيانات كلها: revenue الـ orders لازم يساوي revenue السطور.
27	`recon.order_total_per_order`	الـ orders اللي `order_total` بتاعها مختلف عن مجموع سطورها (من غير سطور بيتحسب 0)	بيلاقي الـ orders بالظبط اللي ورا فشل فحص 26.
28	`recon.orders_count`	الفرق بين صفوف `fact_orders` و `silver_orders` المميزة اللي ليها وقت إنشاء وصف حالة	Gold لازم يحتوي كل order الـ build المفروض يشمله، لا أكتر ولا أقل.
29	`recon.lines_count`	الفرق بين صفوف `fact_order_lines` وصفوف `silver_order_items` اللي الـ order بتاعها في `fact_orders`	مفيش سطر order يضيع أو يتخترع بين Silver و Gold.
30	`recon.attribution_two_rows`	الـ orders اللي ملهاش بالظبط صف `last_touch` وصف `first_touch`	الـ model-ين لازم يغطوا كل order، عشان نقدر نقارن مجاميعهم.
31	`recon.attribution_order_total`	صفوف attribution اللي `order_total` فيها بيختلف عن `fact_orders.order_total`	الـ revenue المنسوب للـ campaigns لازم يساوي revenue الـ orders.
32	`recon.campaign_is_last_touch`	الـ orders اللي `fact_orders.campaign_id` فيها بيختلف عن الـ campaign بتاع last-touch للـ order ده	الـ campaign الظاهر على الـ order لازم يكون last-touch campaign، عشان التقارير تتفق مع بعض.

4. المجموعة 3: Silver hygiene (فحوصات 33 لـ 39)

ببساطة: الـ Silver validator زي حارس البوابة. الفحوصات دي بتسأل: فيه حد وحش عدّى من الحارس؟ نظريًا كلهم بيرجّعوا 0 لأن الـ validator بيحجز الصفوف الوحشة في quarantine. القيمة غير الصفر معناها إن قاعدة في الـ validator ناقصة أو بايظة.

#	الفحص	بيعدّ إيه	ليه مهم
33	`silver.events_null_key_or_time`	صفوف `silver_events` اللي `event_id` أو `event_time` فيها NULL	الـ event من غير id مينفعش يتشال تكراره، ومن غير وقت مينفعش يتحط على يوم.

#	الفحص	بيعدّ إيه	ليه مهم
34	`silver.events_future_time`	events أبعد من 5 دقايق في المستقبل	الـ validator بيسامح 5 دقايق فرق ساعة وبيحجز أي حاجة أبعد في quarantine (`FUTURE_TIMESTAMP`). لو فيه حالة معناها إن القاعدة دي فشلت.
35	`silver.events_unknown_product`	events فيها `product_id` مش موجود في `silver_products` (NULL مسموح)	الـ product المجهول بيتحجز في quarantine (`UNKNOWN_PRODUCT`). لو فيه حالة هتبوّظ تقارير الـ products.
36	`silver.order_status_not_allowed`	صفوف حالة الـ order اللي الحالة فيها مش من حالات عقد Silver	حالة مكتوبة غلط هتبوّظ ترتيب دورة حياة الـ order.
37	`silver.order_items_line_total`	سطور order اللي فيها `line_total` بيختلف عن `quantity x unit_price` بأكتر من 0.01	حساب السعر لازم يطلع مظبوط. هامش EGP 0.01 للتقريب.
38	`silver.snapshots_clicks_gt_impressions`	ad snapshots فيها clicks أكتر من impressions	بيانات مستحيلة. الـ ad report validator بيرفضها (`CLICKS_GT_IMPRESSIONS`).
39	`silver.snapshots_negative_spend`	ad snapshots فيها spend سالب	بيانات مستحيلة. الـ validator بيرفضها (`NEGATIVE_SPEND`).

5. المجموعة 4: Hierarchy and history (فحوصات 40 لـ 46)

5.1 District جوه city (فحوصات 40 لـ 42)

ببساطة: مينفعش يبقى فيه "المعادي، إسكندرية". المعادي في القاهرة.

كل فحص بيعدّ الصفوف اللي الـ district بتاعها مش من districts الـ city بتاعتها. الأزواج المسموحة جاية من نفس ملف الـ simulator (`contracts/geo.py`). الـ district الفاضي، و `unknown` ، والـ city المجهولة مش مخالفات.

#	الفحص	الجدول	ليه مهم
40	`district.silver_customers`	`silver_customers`	customer في "Maadi, Alexandria" هيبقى خطأ بيانات. تعيين الـ store بيعتمد على الـ district.
41	`district.silver_orders`	`silver_orders`	عنوان التوصيل للـ order لازم يكون متسق.
42	`district.silver_events`	`silver_events`	تحليل الـ demand والـ out-of-stock بيستخدم الـ district لاختيار الـ store.

5.2 Slowly changing dimensions (فحوصات 43 لـ 46)

ببساطة: لما سعر منتج أو مدينة عميل تتغيّر، بنضيف نسخة جديدة وبنفضل نعرف النسخة القديمة كانت سارية من إمتى لإمتى. شرطين لازم يتحققوا دايمًا.

`dim_customer` و `dim_product` بيحتفظوا بالتاريخ. كل تغيير بيكتب نسخة جديدة بـ `valid_from` و `valid_to` و `is_current`.

#	الفحص	بيعدّ إيه	ليه مهم
43	`scd2.dim_customer.one_current`	الـ customers اللي ملهمش بالظبط صف `is_current` واحد	صفر صف حالي معناه إن الـ customer اختفى من التقارير الحالية. اتنين معناه إنه بيتعدّ مرتين.
44	`scd2.dim_customer.no_overlap`	أزواج نسخ نفس الـ customer اللي فترات صلاحيتها متداخلة	في أي لحظة الـ customer ليه نسخة واحدة، فالـ fact بيلاقي صف dimension واحد.
45	`scd2.dim_product.one_current`	الـ products اللي ملهاش بالظبط صف `is_current` واحد	نفس الكلام للـ products.
46	`scd2.dim_product.no_overlap`	أزواج نسخ الـ product اللي فترات صلاحيتها متداخلة	نفس الكلام. تغيير السعر مينفعش يسيب سعرين ساريين في نفس الوقت.

6. المجموعة 5: Reported (فحوصات 47 لـ 64)

دي فحوصات من نوع report. مبتفشّلش الـ batch أبدًا. إنسان بيقراها عشان يحكم على التشغيل.

6.1 النسب والحالات الشاذة (فحوصات 47 لـ 52)

#	الفحص	بيبلّغ عن إيه	إزاي تقراه
47	`report.late_rate`	نسبة صفوف `silver_events` اللي اتعلّمت `is_late` ، من 0 لـ 1	حد الـ alert هو 0.05. الـ history replay متأخر بالقاعدة، فالقيمة بتبقى عالية فيه.
48	`report.quarantine_share`	الصفوف المحجوزة مقسومة على quarantine زائد events زائد orders زائد صفوف order status، من 0 لـ 1	حد الـ alert هو 0.02. نسبة صغيرة ثابتة متوقعة، لأن الـ simulator بيحقن صفوف وحشة قصدًا.
49	`report.negative_delivery_lead_hours`	الـ orders اللي `delivery_lead_hours` بتاعتها سالب (اتسلّمت قبل ما تتعمل)	المفروض 0. القيمة فوق 0 بتشاور على مشكلة ساعة أو ترتيب.
50	`report.cancelled_with_shipment`	الـ orders المحسوبة cancelled وفيها كمان صف حالة `order_shipped`	الإلغاء بعد الشحن بيتحجز في quarantine، فده المفروض يبقى 0 عادةً.
51	`report.open_over_14_days`	الـ orders اللي اتعملت من أكتر من 14 يوم ومش delivered ولا returned ولا cancelled	orders عالقة. بتستخدم نفس قاعدة `orders_open` في `agg_campaign_daily` .
52	`report.direct_attribution_share`	نسبة الـ orders اللي ملهاش campaign في الـ model الـ last-touch، من 0 لـ 1	قد إيه revenue مفيش campaign يقدر يدّعيه (direct، ترافيك طبيعي).

6.2 أعداد الصفوف (فحوصات 53 لـ 64)

فحص واحد لكل جدول Gold بيبلّغ عن عدد صفوفه: `rows.dim_customer` و `rows.dim_product` و `rows.dim_campaign` و `rows.dim_date` و `rows.fact_orders` و `rows.fact_order_lines` و `rows.fact_attribution` و `rows.agg_orders_daily` و `rows.agg_funnel_daily` و `rows.agg_campaign_daily` و `rows.agg_city_product_daily` و `rows.customer_360` . ومرقّمين من 53 لـ 64 بالترتيب ده.

أعداد الصفوف مبتفشّل حاجة. هي بتدّي الإنسان رقم يقارنه بالمتوقع بعد التشغيل: الجدول اللي فجأة بقى فاضي أو أكبر عشر مرات سهل تلاحظه.

---

7. المجموعة 6: Manifest (فحصين 65 و 66)

ببساطة: الـ simulator بيحقن أخطاء قصدًا وبيكتب في ورقة (الـ manifest) كام غلطة حطّ. بعدين بنسأل الـ pipeline: لقيت كام؟ لو الرقمين اتطابقوا، يبقى الـ validator شغّال مظبوط.

الـ simulator بيحقن أخطاء قصدًا وبيكتب عدد كل نوع في `noise_manifest.json`. الـ entry-ين دول بيقارنوا الـ pipeline بالـ manifest ده. بيشتغلوا بس لما الـ build ياخد `--manifest`.

#	الفحص	النوع	بيختبر إيه
65	`manifest.quarantine_count`	fail	عدد الصفوف في `silver_quarantine` لازم يكون في حدود 2% من الأخطاء المحقونة اللي المفروض الـ validator يرفضها. الأنواع التسعة هي: `malformed_json` و `missing_field` و `unknown_type` و `bad_version` و `negative_price` و `future_timestamp` و `null_product_id` و `unknown_product_id` و `cancel_after_ship`. التسامح نسبي للعدد المتوقع، والعدد المتوقع 0 بيسمح بـ 0 بالظبط.
66	`manifest.duplicates`	report	الـ manifest بيقول كام event مكرر اتحقن. الـ Silver job بيشيلهم قبل ما يوصلوا النسخة وميحتفظش بعدد، فالرقم ده مينفعش يتقارن. الفحص بيقول "not checkable on the mirror". التكرارات بتتفحص بطريقة تانية: `scripts/check_duplicates.py` وفحوصات 1 لـ 7.

فحص 65 بيثبت إيه؟ لو الـ pipeline فوّت أخطاء، الـ quarantine هيحتوي صفوف أقل من المتوقع. ولو الـ validator رفض صفوف سليمة، هيحتوي أكتر. التطابق معناه إن الـ validator مش بيفوّت أخطاء ومش بيرفض صفوف سليمة. وبيشتغل لأن الـ simulator بتاعنا، فعارفين العدد الحقيقي. مكانش هيشتغل على بيانات حقيقية.

---

8. نتيجة تشغيل 13 أكتوبر 2026

Databricks scratch catalog: Gold build and data quality checks (13 October 2026)

```
PS ...> docker compose run --rm spark python scripts/run_batch.py --skip-publish --full --manifest data/smoke/noise_manifest.json
13 Gold tables of catalog workspace emptied
applied 27 statements (catalog workspace, volume silver_upload)
check dup_key.silver_events                     keys              ok     value=0
check dup_key.fact_orders                       keys              ok     value=0
check recon.order_total_sum                     reconciliation    ok     value=0
check recon.order_total_per_order               reconciliation    ok     value=0
check recon.orders_count                        reconciliation    ok     value=0
check recon.lines_count                         reconciliation    ok     value=0
check recon.attribution_two_rows                reconciliation    ok     value=0
check recon.attribution_order_total             reconciliation    ok     value=0
check recon.campaign_is_last_touch              reconciliation    ok     value=0
check silver.events_unknown_product             silver_hygiene    ok     value=0
check district.silver_customers                 hierarchy_history ok     value=0
check district.silver_orders                    hierarchy_history ok     value=0
check district.silver_events                    hierarchy_history ok     value=0
check scd2.dim_customer.one_current             hierarchy_history ok     value=0
check scd2.dim_customer.no_overlap              hierarchy_history ok     value=0
check scd2.dim_product.one_current              hierarchy_history ok     value=0
check scd2.dim_product.no_overlap               hierarchy_history ok     value=0
check manifest quarantine: expected 308, observed 308, tolerance 2% -> ok
checks: 66 run, 0 failed
BATCH OK in 175.6s
```

الـ Gold build على الـ 66 ، `BATCH OK` : `workspace` فحص، صفر فشل. scratch catalog

البند	القيمة
النتيجة	175.6 ، `BATCH OK` ثانية
الفحوصات اللي اشتغلت	66، منهم صفر فشلوا
مقارنة الـ manifest	308 صف quarantine متوقع، 308 اتلاقوا
الـ Catalog	الـ `workspace` scratch catalog، مش الـ catalog الحقيقي `funnelstream`

أول smoke run كامل في نفس اليوم فشل في المقارنة دي (308 متوقع، 15,472 اتلاقوا). السبب إن نسخة الـ mirror اللي insert-only احتفظت بصفوف من تشغيلات قديمة في الـ scratch catalog. سكريبت الاختبار دلوقتي بينشر بـ `--reset`، وفي سكريبت بيفضّي جداول Gold في الـ scratch. الفحص كان صح: هو اللي مسك البيانات القديمة.

---

9. اللي الفحوصات مبتغطّيهوش

- **فحوصات الـ report مبتفشلش.** 20 من الـ 66 بيطبعوا أرقام بس.
- **التكرارات قبل Silver** مش بتتعدّ. Bronze ممكن يحتوي رسايل اتبعتت تاني. Silver محمي بالـ keys والـ `MERGE`، وفحوصات 1 لـ 7 بتأكّد ده.
- **فحص الـ manifest بيحتاج الـ simulator.** على بيانات حقيقية مفيش قايمة بالأخطاء المحقونة.
- **الفحوصات بتشتغل على نسخة Databricks و Gold،** بعد الـ batch. المسار اللحظي ليه حماياته: alerts الـ late-event والـ quarantine، والـ Silver heartbeat.
- **النجاح مش دليل على الحقيقة.** الفحوصات بتثبت إن الجداول متسقة مع بعضها ومع القواعد، مش إن أرقام البيزنس صح. كل البيانات محاكاة.

---

10. أسئلة متوقعة

هل فحص واحد فاشل بيوقّف كل حاجة؟ فحص من نوع fail قيمته غير صفر بيفشّل الـ batch، والنتيجة بتعرض كل الفحوصات عشان الفشل يبان. أما الخطأ التقني، زي query ميقدرش يشتغل، فبيوقّف الـ build فورًا.

ليه بعض الفحوصات report بس؟ النسب زي الـ late rate وأعداد الصفوف ملهاش قيمة صح ثابتة. لازم إنسان يحكم عليها، فبتتطبع ومبتفشّلش الـ batch.

لو فحص 65 فشل؟ يا إما الـ validator فوّت أخطاء أو رفض صفوف سليمة، يا إما الـ quarantine فيه صفوف قديمة من تشغيل سابق. يوم 13 أكتوبر كانت صفوف قديمة، والحل كان النشر بـ `--reset`.

عرفتوا منين إن الفحوصات نفسها صح؟ `tests/test_checks.py` بيفحص (lint) كل query وبيختبر مقارنة الـ manifest وقاعدة التسامح وشكل النتيجة.